SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	VAIBHAVA LAKSHMI K	**Reg. No.:**	192511310
Section / Batch:	2025 - 2029	**Date:**	22-10-2026

Code of Conduct

I VAIBHAVA LAKSHMI K (192511310) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: VAIBHAVA LAKSHMI K

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
2. Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**
3. Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
4. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q5**

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided

Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Scenario Based Assignment

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes* A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.
(c) Analyze the impact of heuristic accuracy on both algorithms.
(d) Discuss how dynamic changes (blocked paths) affect search performance.
(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Question 2:

Topic:

Smart City Traffic Signal Optimization

Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption.
- Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

Question 3:

Topic:

(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Question 4:

Topic:

Autonomous Rover Exploring Mars

Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

Question 5:

Topic:

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Question 6:

Topic:

University Exam Timetable Generation

Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

Question 7:

Topic:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Question 8:

Topic:

Strategic Game AI for Real-Time Decision Making

Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.
(b) Analyze the computational challenges of large game trees.
(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.
(d) Design an evaluation function and justify the factors considered.
(e) Recommend a strategy for balancing optimality and real-time performance.

ANSWERS

QUESTION 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed Search Strategies – Greedy Best-First Search and A* (a) Greedy Best-First Search: behaviour, strengths and weaknesses

Greedy Best-First Search selects the frontier node that appears closest to the destination using a heuristic. Here, the drone uses straight-line distance.

Evaluation function: $f(n) = h(n)$

Here, $h(n)$ is the estimated distance from node n to the destination.

• Strengths: simple to implement, often explores fewer nodes, and can quickly identify a promising route.

• Weaknesses: ignores the cost already incurred, may choose a short route through flooded or dangerous terrain, and does not guarantee the least-cost route. Incomplete or outdated information can lead to unsafe decisions.

(b) How A* handles the scenario

A* considers both the actual cost of reaching a node and the estimated remaining

cost. Evaluation function: $f(n) = g(n) + h(n)$

$g(n)$ is the actual cost from the start to node n; $h(n)$ estimates the remaining cost; $f(n)$ estimates the total path cost. A longer route with clear roads may be preferred over a shorter route with severe flooding. A* finds an optimal least-cost route when its cost model is appropriate and its heuristic is admissible.

(c) Impact of heuristic accuracy

• Greedy Best-First Search depends heavily on the heuristic, so an inaccurate straight-line estimate may direct the drone toward blocked roads.

• A* also considers the cost already incurred, so it is less dependent on the heuristic alone.

• An admissible heuristic does not overestimate the true minimum remaining cost and helps A* preserve optimality.

• A poor heuristic may increase the number of nodes explored and delay delivery.

The heuristic should use available terrain, weather and obstacle information wherever possible.

(d) Impact of dynamic changes

Sudden flooding or blocked roads can invalidate the current route. The drone should receive updated sensor information, mark blocked paths as unavailable, update route costs, recalculate a path from its current position, and verify that the next movement is safe. Frequent changes increase computational effort; incremental search techniques can reduce this effort by reusing earlier calculations.

(e) Recommendation

Use A* with real-time updates. A* balances the cost already incurred with the estimated remaining cost and can account for travel time, terrain difficulty and risk through an appropriate cost function. Blocked paths should be excluded, sensor information updated regularly, and the drone should replan when conditions change. Safety must not be sacrificed for a small time saving.

QUESTION 2: Smart City Traffic Signal Optimization

Topic: Local Search and Optimization Algorithms (a) Formulation as an optimization problem

Decision variables include green-light duration, signal cycle length, phase order and coordination between neighbouring intersections. The objective is to minimize:

J = w■T + w■F + w■C

T is total waiting time, F is fuel consumption, C is congestion, and w■, w■ and w■ are weights reflecting the importance of each objective. Constraints include safe phase ordering, minimum and maximum green times, pedestrian crossing times and cycle-length limits.

Exhaustive search is inefficient because the number of possible combinations grows rapidly with the number of intersections. Traffic also changes continuously, making a previously calculated solution less effective.

(b) Hill Climbing and its limitations

Hill Climbing starts with an initial timing plan and repeatedly makes a small change, such as extending a green phase. If the change improves the objective, it is retained. This continues until no neighbouring solution improves the score.

• It can become trapped in a local optimum.

• It may stop on a plateau where neighbouring solutions have similar values.

• It may struggle when several intersections must change together.

• Its performance depends on the initial solution.

(c) Local maxima, plateaus and ridges

• **Local optimum:** timings improve nearby intersections but are not best for the whole network.

• **Plateau:** several timing plans produce almost identical waiting times, leaving no clear direction for improvement.

• **Ridge:** improvement requires coordinated changes at several signals; changing only one may not improve traffic, even when changing several together would reduce congestion.

(d) Simulated Annealing and Genetic Algorithms

Simulated Annealing sometimes accepts a worse solution to escape a local optimum. The probability of accepting worse solutions decreases as the temperature cools. Its performance depends on the cooling schedule.

Genetic Algorithm maintains a population of candidate solutions and uses selection, crossover and mutation to generate improved timing plans. It explores multiple candidate solutions, but needs suitable parameter settings and constraint handling.

(e) Recommendation

Use a hybrid Genetic Algorithm and local search approach. A Genetic Algorithm can explore network-wide timing plans, while local search makes rapid adjustments using live traffic information. The system should collect traffic data continuously and enforce safety, pedestrian and signal-timing constraints for every candidate plan.

QUESTION 3: Autonomous Rover Exploring Mars

Topic: Online Search Agent (a) Why an online search agent is required

An online search agent is appropriate because the rover explores unknown terrain without a complete map. Offline search generally plans using an available model before execution, whereas the rover may encounter unexpected rocks, craters or obstacles. It must sense, update its model, plan, act and observe repeatedly.

Sense → Update Model → Plan → Act → Observe

(b) Challenges of partial observability and dynamic environments

• Unknown terrain: the rover may not know what lies beyond its current field of view.

• Sensor limitations: readings may be incomplete or noisy.

• Unexpected obstacles: rocks, craters and difficult terrain may block planned routes.

• Limited resources: energy and communication capacity must be conserved.

• Uncertain movement: actual movement may differ from the expected result.

These challenges require continuous observation, cautious navigation and frequent replanning.

(c) Building and updating the internal model

The rover maintains an internal representation of its estimated position, visited locations, detected obstacles, unexplored regions and movement costs. Sensors collect information; the rover updates its position and local map, marks hazards, assigns terrain costs, selects a feasible route and compares actual movement with predictions. The model is revised whenever new evidence is received.

(d) Comparison with uninformed and informed search

Feature	Uninformed	Informed Search	Online Search

	Search		
Knowledge	No heuristic guidance	Uses heuristic information	Learns while acting
Environment	Usually uses a known model	Uses an available model	Can begin with incomplete model
Decision-making	Explores by a fixed rule	Prioritizes promising states	Alternates action and learning
Adaptability	Needs replanning to adapt	Depends on updates and re	planningSupports epeated updates
Example	Breadth-First Search	A* Search	Agent using local sensing

Online search can also use heuristic guidance; therefore, online and informed search are not mutually exclusive.

(e) Recommendation

Use an online search agent with heuristic-guided planning. The rover should use local sensing to update its map and an A*-style heuristic to prioritize promising routes when enough information is available. It should avoid known hazards, consider energy consumption, replan when obstacles are discovered and maintain a safe fallback when information is uncertain.

QUESTION 4: University Exam Timetable Generation

Topic: Constraint Satisfaction Problem (CSP) (a) CSP formulation: variables, domains and constraints

A CSP consists of variables, domains and constraints.

- • **Variables:** each exam can be represented as X■ = (T■, H■), where T■ is the assigned time slot and H■ is the assigned hall.
- • **Domains:** the available examination slots and eligible halls for each exam, considering capacity, accessibility and facilities.
- • **Constraints:** students cannot have overlapping exams; hall capacity cannot be exceeded; required subject ordering must be respected; faculty availability must be considered where supervision is needed; and students requiring accommodations must receive suitable arrangements.

A valid timetable must satisfy every hard constraint.

(b) Backtracking search

Backtracking assigns values to variables one at a time. It selects an exam, assigns a feasible slot and hall, checks the constraints and proceeds to the next exam. If a conflict occurs or no valid choice

remains, it undoes the most recent assignment and tries another value. It continues until a complete valid timetable is found or the choices are exhausted.

(c) Constraint propagation and forward checking

Constraint propagation reduces domains whenever a new assignment is made. With forward checking, after assigning an exam, the system removes conflicting time slots from the domains of unassigned exams taken by the same students. If any domain becomes empty, failure is detected immediately and the algorithm backtracks.

• Detects conflicts early.

• Reduces unnecessary search.

• Avoids continuing with impossible partial assignments.

• Improves timetable-generation efficiency.

(d) Real-world challenges and scalable strategy

Challenges include many exams and students, limited room capacity, enrolment changes, faculty availability, accessibility needs and conflicting preferences.

A scalable approach combines backtracking with forward checking, the **Minimum Remaining Values (MRV)** heuristic to choose the exam with the fewest available options, and the **Least Constraining Value (LCV)** heuristic to choose the assignment that restricts other exams the least. Hard constraints must always be satisfied; soft preferences can be optimized after a valid timetable is found.

QUESTION 5: Strategic Game AI for Real-Time Decision Making

Topic: Minimax, Alpha-Beta Pruning and Evaluation Functions (a) Minimax algorithm

Minimax models a game as alternating decisions by two opposing players. The AI is the maximizing player and tries to obtain the highest utility, while the opponent is the minimizing player. Terminal states receive utility values such as win, loss or draw. The algorithm propagates values upward: maximizing nodes choose the highest child value and minimizing nodes choose the lowest.

At a maximizing node: V(s) = max■ V(Result(s,a))

At a minimizing node: V(s) = min■ V(Result(s,a))

(b) Computational challenges

If b is the average branching factor and d is search depth, Minimax has approximate time complexity O(b■). The number of states grows exponentially as possible actions and depth increase. This causes high computation time, increased memory requirements and difficulty meeting real-time decision deadlines. Depth limits and an evaluation function are therefore needed.

(c) Alpha-Beta pruning

Alpha-Beta pruning eliminates branches that cannot influence the final decision. Alpha (α) is the best value currently guaranteed to the maximizing player; beta (β) is the best value currently guaranteed to the minimizing player.

When $\alpha \geq \beta$, the remaining children of that branch can be pruned because they cannot improve the decision already available to an ancestor. Alpha-Beta returns the same decision as Minimax when correctly implemented but explores fewer nodes.

• Worst-case time complexity: O(b■).

• Best-case time complexity with favourable move ordering: approximately O(b^(d/2)).

(d) Example evaluation function

For a non-terminal state, one possible evaluation function is:

E(s) = 5M + 3U + 2R + 4O + 2I − 5D

M = difference in military strength; U = difference in useful units; R = resource advantage; O = objective or territory control advantage; I = income or production advantage; D = immediate danger to critical units or the base. These illustrative weights should be tuned through testing, with each feature measured as AI advantage minus opponent advantage.

Military strength estimates combat ability, units reflect remaining forces, resources support future production, objective control measures progress toward victory, income estimates long-term strength and danger helps protect critical assets. Terminal wins and losses should receive values much larger than ordinary positional scores.

(e) Recommendation

Use depth-limited Minimax with Alpha-Beta pruning and iterative deepening. Iterative deepening searches progressively greater depths and retains the best result from the last completed search if the time limit is reached. Good move ordering, a reliable evaluation function, a strict time budget and selective deeper searches for critical situations help balance decision quality and real-time performance.

CONCLUSION

Artificial Intelligence techniques provide effective solutions for complex real-world problems. A* supports cost-aware route planning, local search algorithms optimize traffic signals, online search enables exploration of unknown environments, CSP techniques generate valid examination timetables, and Minimax with Alpha-Beta pruning supports strategic game decisions. The effectiveness of these methods depends on accurate models, suitable heuristics, efficient search, constraint handling and the ability to adapt to changing conditions. Selecting the appropriate algorithm helps AI systems balance efficiency, accuracy and safety.